

Project Architecture & Folder Guide

Comprehensive Technical Reference: Directory Responsibilities, Engineering Rationale & Data Lifecycle

Project: Customer Churn & Retention Analytics

Architecture Pattern: Modular Pipeline & Decoupled Storage

Standard: Human Data Engineering Best Practices

1. Executive Summary: Why Architecture & Folder Structure Matter

A common pitfall in novice data science projects is dumping data, exploratory code, generated graphs, trained models, and final reports into a single cluttered directory. This causes naming collisions, breaks reproducibility, risks accidental overwrites of raw data, and makes the project inaccessible to team members or hiring managers.

This project adopts the **Separation of Concerns (SoC)** architectural standard utilized by senior data engineers and production ML teams. Every directory has a strictly defined single responsibility, creating a transparent, auditable pipeline from raw transactional ingestion to executive decision-making:

Directory / Layer	Primary Function (What It Does)	Engineering Rationale (Why We Create It)
data/	Houses raw inputs and cleansed analytical datasets.	Preserves immutable raw data; prevents irreversible corruption.
src/	Contains modular, testable Python source code.	Eliminates 'spaghetti code'; enables automated CLI pipeline execution.
notebooks/	Step-by-step narrative notebooks (Audit & Modeling).	Facilitates interactive visual storytelling and methodology verification.
models/	Stores serialized ML pipelines (.joblib).	Decouples model training from real-time API or dashboard inference.
visualizations/	Stores high-resolution (300 DPI) exported charts.	Separates visual assets from code; ready for reports and slide decks.
sql/	Enterprise ANSI SQL schema DDL and analytical queries.	Enables direct execution inside Postgres, Snowflake, or BigQuery.
tableau/	Tableau extract and calculated field documentation.	Bridges data science with executive BI dashboard authoring.
reports/	Executive summaries and interactive web applications.	Packages strategic takeaways and ROI models for business stakeholders.

2. End-to-End Data Lifecycle & Flow

Data moves through the repository in a strictly unidirectional flow, ensuring full traceability and zero data leakage:

- **1. Raw Ingestion (data/raw_...csv):** Represents untouched operational records as received from production billing systems.
- **2. Cleansing & Validation (src/data_cleansing.py):** Audits formatting, handles missing values, and writes data/cleaned_...csv.
- **3. Feature Engineering & Modeling (src/train_models.py):** Reads clean data, encodes features, and saves models to models/.
- **4. Asset & Plot Generation (src/eda_visualizations.py):** Exports visual plots to visualizations/ without modifying data.
- **5. Business Reporting (reports/ & generate_pdf_report.py):** Consumes clean data, models, and charts to generate stakeholder artifacts.

3. Deep-Dive: Directory Responsibilities & Engineering Rationale

Folder 1: data/ (The Data Storage Layer)

Component	Detailed Specification
Key Files	raw_customer_churn_data.csv (10k raw uncleaned records) cleaned_customer_churn_data.csv (Generated post-cleansing)
What It Does	Serves as the dedicated repository for data files across different lifecycle states: pristine raw data and certified clean data.
Why We Create It	Principle of Immutability: Raw data must never be manually edited or overwritten. By separating raw from cleaned files, any data scientist can re-run the pipeline from scratch and reproduce identical results. It also prevents production databases from being queried repeatedly.
Handling Defects	The raw dataset deliberately contains realistic defects (266 records with whitespace charges in Total_Charges). The clean dataset is created only after running the cleansing module, proving data hygiene.

Folder 2: src/ (The Production Source Code Layer)

Component	Detailed Specification
Key Modules	data_generator.py, data_cleansing.py, data_cleansing_visualizer.py, feature_engineering.py, eda_visualizations.py, train_models.py, retention_strategy.py, __init__.py
What It Does	Contains modular, object-oriented, and functional Python code powering each analytical phase. Each file is an executable module and importable library.
Why We Create It	Modularity & Testability: Notebooks are great for exploration but terrible for automated deployment. Packaging logic into Python scripts allows unit testing, CLI execution, integration into CI/CD pipelines, and re-use across multiple notebooks without code duplication.
Design Standard	Follows PEP 8 standards with descriptive docstrings, type validation, parameterization, and isolated side effects.

Folder 3: notebooks/ (The Interactive Research Layer)

Component	Detailed Specification
Notebooks	01_data_cleansing_and_audit.ipynb (Data Quality Audit) 02_customer_churn_and_retention.ipynb (Full EDA, ML & Retention Strategy)
What It Does	Provides rich, interactive computational environments combining executable Python code with Markdown narratives, LaTeX math, and inline visualizations.
Why We Create It	Auditing & Scientific Storytelling: Stakeholders and technical evaluators can run code cell-by-cell, observe intermediate tables, and verify hypotheses interactively without running terminal commands. Splitting into two notebooks ensures clean separation between data engineering and modeling.

Folder 4: models/ (The Serialized Artifacts Layer)

Component	Detailed Specification
Key Artifact	best_churn_model.joblib (Starts empty with .gitkeep; saved during training)
What It Does	Stores serialized Scikit-Learn pipelines containing fitted scalers, encoders, and the trained classifier weights.
Why We Create It	Inference Decoupling: Retraining machine learning models is computationally expensive. Persisting models enables instant inference (scoring new customer accounts in milliseconds) for batch jobs or web applications without retraining.

Folder 5: visualizations/ (The Visual Evidence Layer)

Component	Detailed Specification
Exported Assets	data_cleansing_audit.png, churn_by_contract.png, churn_by_support_tickets.png, churn_by_payment_method.png, churn_by_tenure_band.png, model_roc_curves.png, feature_importance.png, confusion_matrices.png, retention_campaign_roi.png
What It Does	Stores publication-ready 300 DPI image assets generated during EDA and model evaluation.
Why We Create It	Asset Decoupling: External documents (PDF reports, executive slide decks, web dashboards) require static image files. Housing charts in a dedicated folder ensures they can be embedded cleanly without re-executing notebooks.

Folder 6: sql/ (The Enterprise Database Layer)

Component	Detailed Specification
Key Script	churn_analysis.sql (ANSI SQL schema, CTEs, Window functions, Cohort retention)
What It Does	Houses enterprise SQL queries that execute churn analytics directly within relational databases or cloud data warehouses.
Why We Create It	Database Native Analytics: Not all analytics run in Python. Data analysts and SQL engineers need direct database queries. This script proves advanced SQL competence (CTEs, ROW_NUMBER(), MIN() OVER(), partitioned sums) for database warehouses.

Folder 7: tableau/ (The Business Intelligence Layer)

Component	Detailed Specification
Key Files	tableau_calculated_fields.md (Calculated fields guide & LOD expressions) tableau_churn_dataset.csv (Enriched flat extract generated during pipeline execution)
What It Does	Provides ready-to-import data extracts and formulas for building interactive BI dashboards in Tableau or PowerBI.
Why We Create It	BI Enablement: Bridges data science with non-technical business users. Business analysts can immediately build dashboards using pre-engineered dimensions without needing to re-implement Python transformation logic.

Folder 8: reports/ (The Stakeholder Delivery Layer)

Component	Detailed Specification
Key Files	executive_summary.md, interactive_dashboard.html, model_benchmarks.json, retention_financial_summary.json
What It Does	Contains executive-facing summaries, machine-readable JSON metrics, and the single-page HTML interactive simulator.
Why We Create It	Actionable Decision Making: Corporate executives care about financial outcomes (MRR lost, CLV saved, campaign ROI) rather than code lines. This folder consolidates high-level business intelligence into easily digestible formats.

4. Root Files & Pipeline Orchestration

The repository root contains five essential configuration and execution files that tie all directories together:

File	Function & Purpose
main.py	Master CLI Orchestrator: Single entry point to run all pipeline steps sequentially (clean, eda, train, strategy, report) or execute individual steps via --step.
generate_pdf_report.py	Automated PDF Compiler: Compiles the 5-page executive report embedding generated charts and financial metrics with auto-prerequisite checking.
customer_churn_analytics.json	Project Blueprint: Defines business objectives, dataset schema, key metrics, and empirical findings to validate.
requirements.txt	Dependency Manifest: Specifies exact library versions (pandas, scikit-learn, xgboost, seaborn, matplotlib, reportlab).
README.md	Project Portfolio Documentation: Explains business context, findings, model comparisons, and human-written execution instructions.

5. Step-by-Step Execution Guide for Evaluators

To experience the project exactly as designed, run the following commands in order from your terminal:

- **Step 1: Install Dependencies**
pip install -r requirements.txt (Installs pandas, scikit-learn, xgboost, seaborn, reportlab).
- **Step 2: Execute Data Cleansing**
python main.py --step clean
Reads data/raw_customer_churn_data.csv, resolves whitespace defects, and generates data/cleaned_customer_churn_data.csv.
- **Step 3: Run Exploratory Data Analysis**
python main.py --step eda
Generates high-res visual plots in visualizations/ verifying the 4x contract risk and 81.7% ticket cliff.
- **Step 4: Train Machine Learning Classifiers**
python main.py --step train
Trains Logistic Regression, Random Forest, and XGBoost; exports models/best_churn_model.joblib.
- **Step 5: Compute Business Retention Strategy**
python main.py --step strategy
Calculates MRR Lost (\$231.9K), CLV (\$1,848), and models the 175.5% ROI campaign.
- **Step 6: Generate Executive PDF Reports**
python main.py --step report (Compiles the 5-page executive intelligence report).